

컴퓨터공학과
클라우드 컴퓨팅(Cloud Computing)
컨테이너 오케스트레이션 - 2 (Kubernetes)

■ 이번 주 실습 순서

1. Kubernetes의 구조와 동작
2. 지난주 앱 이미지 생성과 Compose 복습
3. Minikube 준비와 kubectl 기본 조작
4. Deployment·Service 배포와 기능 확인
5. 확장·복구·업데이트·롤백·정리

■ Kubernetes

컨테이너 오케스트레이션을 위한 플랫폼

Google에서 시작한 오픈소스 프로젝트

CNCF(Cloud Native Computing Foundation)에서 관리
선언한 상태를 기준으로 배포·확장·복구 관리

■ 주요 구성 요소

Node: Pod를 실행하는 컴퓨팅 환경

Pod: 하나 이상의 컨테이너를 묶는 최소 배포 단위

Deployment: Pod 템플릿·복제 수·업데이트 관리

Service: 선택한 Pod로 연결하는 안정된 주소

Namespace: 클러스터 안의 논리적인 작업 영역

■ Control Plane과 Node

Control Plane: 클러스터 상태 관리

API Server: 요청 수신, etcd: 설정과 상태 저장

Scheduler: 노드 선택, Controller: 원하는 상태 유지

Node: kubelet이 Pod 상태를 보고하고 런타임이 실행
컨테이너 런타임의 예: containerd, CRI-O

■ 원하는 상태와 실제 상태

사용자가 YAML에 원하는 Pod 수와 이미지를 선언
kubectl이 API Server에 적용 요청 전달
컨트롤러가 원하는 상태와 실제 상태를 비교
Scheduler가 배치하고 kubelet·런타임이 실행
Pod를 삭제하면 Deployment가 새 Pod 생성

■ 지난주 앱 연결

nginx: HTML 화면 제공과 /api/ 요청 전달

Flask: 프로젝트 검색과 소개글 분량 검사

이번 주 추가: health 응답의 **version**과 **pod**

Compose와 Kubernetes에서 같은 앱·이미지 사용

■ 새 저장소와 작업 폴더

교수자 week6_practice를 본인 계정으로 Fork
본인 작업 폴더에서 명령 실행
이후 모든 명령은 복제한 **week6** 폴더 기준

```
git clone https://github.com/내계정/week6_practice.git week6
Set-Location .\week6
code .
```

■ 실습 환경 확인

Docker Desktop의 Linux 컨테이너 엔진 실행

VS Code 터미널은 **PowerShell** 선택

예상 결과: Docker Client와 Server 버전 모두 표시

```
docker version
docker compose version
kubectl version --client
```

■ 이미지 생성 복습

이미지: 코드·라이브러리·실행 명령을 담은 묶음

Flask는 이미지 안에 설치하므로 Windows 설치 불필요

두 이미지를 생성하고 태그 확인

```
docker build -t week6-api:v1 ./api
docker build -t week6-web:v1 ./nginx
docker image ls
```

예상 결과: *week6-api:v1, week6-web:v1*

■ 학교 인증서 오류 구분

기본 Dockerfile은 TLS 인증서를 검증

pip 설치 오류와 Docker 이미지 pull 오류를 구분

정상 해결: IT 담당자가 확인한 CA 번들 사용

Dockerfile.school은 학교 실습 한정 별도 선택

trusted-host는 중간자 공격·패키지 변조 위험 존재

■ 학교 실습 한정 빌드

교수자가 학교의 인증서 문제임을 확인한 경우만 사용
PyPI 두 호스트에만 trusted-host 적용
requirements.txt의 **Flask==3.1.2** 고정 유지

```
docker build -f ./api/Dockerfile.school -t week6-api:v1 ./api
```

■ Compose 실행과 확인

compose.yaml: api와 nginx 두 서비스

예상 결과: HTTP 200, status ok, version v1

브라우저 localhost:8080에서 검색·분량 검사

```
docker compose config
docker compose up -d
docker compose ps
curl.exe -i http://localhost:8080/api/health
```

■ Compose 로그와 정리

502이면 api 준비 상태와 로그 확인

Kubernetes에서 8080을 쓰기 전에 Compose 종료

down 후에도 이미지는 남음

```
docker compose logs --tail 10 api nginx  
docker compose down
```

■ Minikube

로컬 PC에 학습용 Kubernetes 클러스터 구성

Docker 드라이버로 학습용 노드 실행

Windows Minikube 사용, Ubuntu 추가 설치 불필요

Docker Desktop 내장 Kubernetes는 별도 활성화 불필요

■ Minikube 설치 확인

설치되어 있으면 설치 단계를 반복하지 않음
학교 제공 검증 파일 또는 공식 설치 안내 사용
winget 설치 후 새 PowerShell 터미널 열기

```
winget install Kubernetes.minikube  
minikube version
```

■ 실습 클러스터 시작

이번 주 전용 프로필 **week6** 사용

최초 실행은 노드·Kubernetes 이미지 다운로드 필요

예상 결과: 노드 상태 **Ready**

```
minikube start -p week6 --driver=docker --cpus=2 --memory=3072
```

```
minikube status -p week6
```

```
kubectl --context=week6 get nodes
```

■ kubectl 버전 맞추기

Docker Desktop의 kubectl이 서버보다 오래될 수 있음
Minikube가 제공하는 호환 버전을 현재 터미널에서 사용
새 터미널 B에서도 함수 한 줄을 다시 실행

```
function kubectl { minikube kubectl -p week6 -- @args }  
kubectl version --client  
kubectl --context=week6 get nodes
```

▪ kubectl 기본 명령

get: 리소스 목록과 상태

describe: 세부 설정·이벤트·실패 원인

logs: 컨테이너 출력 로그

apply -f: YAML 적용, delete: 지정 리소스 삭제

명령 구조: kubectl 명령 리소스 이름 옵션

■ 로컬 이미지 전달

호스트 Docker 이미지와 노드 이미지 저장소는 별개

image load로 만든 이미지를 노드에 전달

예상 결과: v1 이미지 두 개가 목록에 표시

```
minikube image load week6-api:v1 -p week6
minikube image load week6-web:v1 -p week6
minikube image ls -p week6
```

■ Namespace와 YAML

Namespace는 이번 수업 자원을 구분하는 작업 영역
YAML 들여쓰기는 공백, ---는 리소스 구분
첫 조회의 No resources found는 정상

```
kubectl --context=week6 apply -f ./k8s/00-namespace.yaml  
kubectl --context=week6 -n week6 get pods
```

▪ Deployment 설정

replicas: 원하는 Pod 수

template: 새 Pod의 컨테이너·이미지 설정

selector와 template.labels는 같은 대상을 가리킴

imagePullPolicy Never: 이번 실습은 로컬 이미지 사용

readinessProbe: 요청을 받을 준비 상태 확인

■ API 배포

먼저 api Service와 Flask Pod 준비

예상 결과: deployment api successfully rolled out

다음 단계의 nginx가 api:5000으로 연결

```
kubectl --context=week6 apply -f ./k8s/api.yaml
```

```
kubectl --context=week6 -n week6 rollout status deployment/api --timeout=120s
```

실패하면 describe pod와 logs deployment/api로 원인을 확인합니다.

■ 웹 배포와 상태 확인

API가 준비된 뒤 nginx 배포

예상 결과: api와 web이 각각 **1/1 Running**

Deployment와 Service가 각각 두 개 존재

```
kubectl --context=week6 apply -f ./k8s/web.yaml
```

```
kubectl --context=week6 -n week6 rollout status deployment/web --timeout=120s
```

```
kubectl --context=week6 -n week6 get "deployments,pods,services"
```

▪ Service와 포트

Pod는 교체되면 이름·IP가 바뀔 수 있음

Service는 selector로 준비된 Pod를 선택

api Service 5000: Flask 5000으로 연결

web Service 80: nginx 80으로 연결

브라우저 8080은 port-forward로 연결

■ Service 유형

ClusterIP: 클러스터 내부의 안정된 주소

NodePort: 노드의 지정 포트로 접근

LoadBalancer: 외부 로드밸런서 구현과 연동

이번 실습: ClusterIP와 localhost port-forward 사용

■ 웹 접속용 터미널 A

아래 명령을 실행한 채 터미널 유지

예상 결과: Forwarding from 127.0.0.1:8080

브라우저에서 <http://localhost:8080> 접속

```
kubectl --context=week6 -n week6 port-forward service/web 8080:80
```

■ 기능 확인용 터미널 B

새 PowerShell 터미널을 열어 week6 폴더 확인
health의 pod 값을 get pods 결과와 비교
검색·분량 검사 기능이 Compose 때와 같은지 확인

```
curl.exe -i http://localhost:8080/api/health  
kubectl --context=week6 -n week6 get pods  
kubectl --context=week6 -n week6 logs deployment/api --tail=10
```

Kubernetes 실습

■ Kubernetes 앱 실행 화면

검색어 캡스톤: 1개 프로젝트
소개글 106자: 권장 분량 충족

WEEK 6 - KUBERNETES PRACTICE

캡스톤 기획 도우미

프로젝트 예시를 찾고 소개글 분량을 확인합니다.

기본 연결 확인: </api/health>

프로젝트 검색

분야

전체

검색어

캡스톤

검색

1개 프로젝트

캡스톤 팀원 모집 (web) — 관심 분야와 기술에 맞는 팀원을 찾습니다.

소개글 분량 검사

프로젝트 소개글

이 프로젝트는 학교의 공간 예약 현황을 쉽게 확인하도록 돕는 가상의 웹 서비스입니다. 사용자는 원하는 날짜와 시간을 선택해 빈 강의실을 확인하고, 예약 가능 여부를 화면에서 볼 수 있습니다.

검사

공백 포함 106자 / 공백 기준 27단어 / 권장 100~300자: 충족

분량만 확인합니다. 글의 품질을 평가하지 않습니다.

nginx는 화면을 제공하고 Flask는 검색과 분석을 처리합니다.

■ API 복제본 확대

명령으로 원하는 복제 수를 **3**으로 변경

예상 결과: api Pod 3개가 준비됨

이 단계는 부하와 관계없는 **수동 확장**

```
kubectl --context=week6 -n week6 scale deployment/api --replicas=3
kubectl --context=week6 -n week6 rollout status deployment/api --timeout=120s
kubectl --context=week6 -n week6 get pods -l app=api
```

replicas: 1인 YAML을 다시 apply하면 1개로 되돌아갑니다.

■ Pod 삭제와 자동 복구

get pods에서 api Pod 이름을 복사
아래 POD이름을 실제 이름으로 바꿔 실행
새 이름의 Pod가 생겨 최종 3개인지 확인

```
kubectl --context=week6 -n week6 delete pod POD이름  
kubectl --context=week6 -n week6 get pods -l app=api -w
```

■ 새 버전 이미지 생성

같은 태그를 덮어쓰지 않고 **v2** 생성

APP_VERSION 값으로 health 응답 변화 확인

빌드한 v2도 노드에 전달

```
docker build --build-arg APP_VERSION=v2 -t week6-api:v2 ./api
minikube image load week6-api:v2 -p week6
```

■ 이미지 업데이트

Deployment의 api 컨테이너를 v2로 변경

롤아웃 완료 후 health 확인

예상 결과: version **v2**, 새로운 Pod 이름

```
kubectl --context=week6 -n week6 set image deployment/api api=week6-api:v2  
kubectl --context=week6 -n week6 rollout status deployment/api --timeout=120s  
curl.exe -s http://localhost:8080/api/health
```

set image는 로컬 YAML 파일을 바꾸지 않습니다.

■ 이전 버전으로 롤백

직전 Pod 템플릿 revision으로 되돌림

예상 결과: health version **v1**

Git 파일과 저장 데이터까지 되돌리는 명령은 아님

```
kubectl --context=week6 -n week6 rollout undo deployment/api  
kubectl --context=week6 -n week6 rollout status deployment/api --timeout=120s  
curl.exe -s http://localhost:8080/api/health
```

■ 실행 오류 확인

ErrImageNeverPull: 태그와 image load 확인

Pending: describe 이벤트에서 자원 부족 확인

CrashLoopBackOff: logs --previous로 이전 로그 확인

nginx 502: API 상태·Service selector·EndpointSlice 확인

접속 끊김: 터미널 A의 port-forward 상태 확인

■ 실습 자원 정리

터미널 A: Ctrl+C로 port-forward 종료

터미널 B: 이번 주 namespace 삭제와 프로필 중지

예상 결과: namespace 조회 시 **NotFound**

```
kubectl --context=week6 delete namespace week6 --wait=true  
kubectl --context=week6 get namespace week6  
minikube stop -p week6  
docker compose down
```

■ 다음 실습과 완전 삭제

재개: week6 프로필 시작 후 이미지·YAML 단계 진행
더 필요 없는 경우에만 week6 프로필 삭제
프로필 삭제는 그 안의 모든 데이터를 삭제

```
minikube start -p week6 --driver=docker  
# 실습 종료 후 완전 삭제가 필요한 경우만 실행  
minikube delete -p week6
```

■ 6주차 과제

가상의 프로젝트 1개를 추가한 v2 이미지 배포
검색 결과와 version 변화 기록
API Pod 3개 중 하나를 삭제하고 복구 관찰
v1 롤백 후 검색 결과와 version 비교
명령·결과 스크린샷과 저장소 링크·커밋 제출

■ 학습 확인

이미지 생성과 Pod 생성의 차이는 무엇인가요?

Service selector와 Pod label은 왜 같아야 하나요?

Pod를 삭제했는데 다시 생기는 이유는 무엇인가요?

수동 확장과 HPA의 차이는 무엇인가요?

롤백 후 코드 파일도 이전 상태로 바뀌나요?



END